SQL FOR DATA ANALYSIS · LESSON 16 OF 20

Recursive CTEs, Hierarchies and Number Series

Use recursive common table expressions to walk hierarchies such as org charts and category trees, build number and date series without a calendar table, and avoid runaway recursion.

Gap-fill lesson | 2026 | Advanced | 120 min

By the end of this lesson you can

- Explain the anchor member, the recursive member and the stopping condition of a recursive CTE
- Walk an org chart or category tree and report each node's level and full path
- Generate number and date series inside SQL
- Protect queries against infinite recursion

Datasets used

`superstore_sales.csv`

Why hierarchies need recursion

Many business tables describe a tree. An employee reports to a manager who reports to a director. A product category contains sub-categories, which contain products. A bill of materials lists parts that are themselves made of parts. A plain `JOIN` only goes one level deep. A **recursive CTE** keeps joining a table to its own results until no new rows appear, so it can walk any depth.

CASE STUDY **Who reports to whom?**

A regional head wants to know, for every employee, how many management levels sit above them and who their top-level leader is, so that a re-organisation can be planned. The org chart is a single table where each row names its manager. One query has to follow that chain to the top, whatever its length.

Anatomy of a recursive CTE

Every recursive CTE has three parts:

1. **Anchor member**: the starting rows, for example the people with no manager.
2. **Recursive member**: a query that joins back to the CTE itself to find the next level.
3. **Stopping condition**: the recursion ends when the recursive member returns no rows.

```
WITH RECURSIVE name AS (
    SELECT ...            -- anchor: level 0
    UNION ALL
    SELECT ...            -- recursive: joins name to the next level
    FROM table t
    JOIN name n ON ...
)
SELECT * FROM name;
```

`UNION ALL` is the usual choice, because it does not remove duplicates and so runs faster. Use `UNION` only when you need the engine to drop repeated rows, which also protects against cycles (see the warning below). In SQL Server the keyword `RECURSIVE` is not written; in PostgreSQL, DuckDB and SQLite it is.

Example 1: an org chart

The table below is self-contained so you can run it as written. Each person has at most one manager; the CEO has no manager.

SQL

```sql
WITH RECURSIVE org(emp_id, name, manager_id) AS (
    VALUES
        (1, 'Anika',  NULL),
        (2, 'Rohit',  1),
        (3, 'Meera',  1),
        (4, 'Kabir',  2),
        (5, 'Tara',   2),
        (6, 'Dev',    3),
        (7, 'Isha',   4)
),
chain AS (
    -- Anchor: the top of the hierarchy
    SELECT emp_id, name, manager_id, 0 AS level, name AS path
    FROM org
    WHERE manager_id IS NULL
    UNION ALL
    -- Recursive: everyone whose manager is already in the chain
    SELECT o.emp_id, o.name, o.manager_id, c.level + 1, c.path || ' > ' || o.name
    FROM org o
    JOIN chain c ON o.manager_id = c.emp_id
)
SELECT level, name, path
FROM chain
ORDER BY path;
```

RESULT

level	name	path
0	Anika	Anika
1	Meera	Anika > Meera
2	Dev	Anika > Meera > Dev
1	Rohit	Anika > Rohit
2	Kabir	Anika > Rohit > Kabir
3	Isha	Anika > Rohit > Kabir > Isha
2	Tara	Anika > Rohit > Tara

7 rows × 3 columns

Read the result as a tree: level 0 is the CEO, level 1 the direct reports, and so on. The `path` column is built on the way down by appending each name to the parent's path, which is a common pattern for showing a breadcrumb.

Key idea: the recursive member can use anything the anchor produced, here `c.level` and `c.path`. That is how depth and paths are accumulated.

Example 2: team totals across the whole tree

Once you can attach a root and a level to every node, you can roll up numbers. Here every person's team size counts everyone below them.

SQL

```
WITH RECURSIVE org(emp_id, name, manager_id) AS (
    VALUES (1,'Anika',NULL),(2,'Rohit',1),(3,'Meera',1),(4,'Kabir',2),(5,'Tara',2),
(6,'Dev',3),(7,'Isha',4)
),
below AS (
    -- Each person paired with every person who reports to them, at any depth
    SELECT manager_id AS boss, emp_id AS report FROM org WHERE manager_id IS NOT NULL
    UNION ALL
    SELECT b.boss, o.emp_id
    FROM below b
    JOIN org o ON o.manager_id = b.report
)
SELECT o.name, COUNT(b.report) AS people_below
FROM org o
LEFT JOIN below b ON b.boss = o.emp_id
GROUP BY o.name
ORDER BY people_below DESC, o.name;
```

RESULT

name	people_below
Anika	6
Rohit	3
Kabir	1
Meera	1
Dev	0
Isha	0
Tara	0

7 rows × 2 columns

Notice that `below` is built by walking down from each boss. `COUNT` then counts every report at every depth, which a single `JOIN` could not do.

Example 3: category trees with depth

Product catalogues often nest categories several levels deep. The example below stores a small tree as a parent-child list and returns each node with its depth, so that a report can indent the categories.

SQL

```sql
WITH RECURSIVE cat(cat_id, cat_name, parent_id) AS (
    VALUES
        (1, 'Technology', NULL),
        (2, 'Phones', 1),
        (3, 'Accessories', 1),
        (4, 'Chargers', 3),
        (5, 'Cables', 3),
        (6, 'Office Supplies', NULL),
        (7, 'Paper', 6)
),
tree AS (
    SELECT cat_id, cat_name, 1 AS depth, cat_name AS full_path
    FROM cat WHERE parent_id IS NULL
    UNION ALL
    SELECT c.cat_id, c.cat_name, t.depth + 1, t.full_path || ' / ' || c.cat_name
    FROM cat c
    JOIN tree t ON c.parent_id = t.cat_id
)
SELECT depth, REPEAT('    ', depth - 1) || cat_name AS indented_name, full_path
FROM tree
ORDER BY full_path;
```

RESULT

depth	indented_name	full_path
1	Office Supplies	Office Supplies
2	Paper	Office Supplies / Paper
1	Technology	Technology
2	Accessories	Technology / Accessories
3	Cables	Technology / Accessories / Cables
3	Chargers	Technology / Accessories / Chargers
2	Phones	Technology / Phones

7 rows × 3 columns

The REPEAT call indents the name by its depth, which is the simplest way to show a tree in a flat table or a report.

Example 4: a number and date series

Recursive CTEs can create rows out of nothing. This is useful for building a list of months so that a month with zero sales still appears in a report, rather than vanishing because no order was placed.

SQL

```sql
WITH RECURSIVE months(month_start) AS (
    SELECT DATE '2024-01-01'
    UNION ALL
    SELECT month_start + INTERVAL 1 MONTH
    FROM months
    WHERE month_start < DATE '2024-12-01'
),
monthly_sales AS (
    SELECT DATE_TRUNC('month', order_date) AS month_start, SUM(sales) AS sales
    FROM superstore
    WHERE order_date ⩾ DATE '2024-01-01' AND order_date < DATE '2025-01-01'
    GROUP BY 1
)
SELECT m.month_start,
       COALESCE(s.sales, 0) AS sales
FROM months m
LEFT JOIN monthly_sales s ON s.month_start = m.month_start
ORDER BY m.month_start;
```

RESULT

month_start	sales
2024-01-01	6,022,738
2024-02-01	2,463,147
2024-03-01	5,940,384
2024-04-01	4,620,479
2024-05-01	5,253,017
2024-06-01	4,012,674
2024-07-01	4,818,164
2024-08-01	5,757,228
2024-09-01	5,330,618
2024-10-01	7,255,348
2024-11-01	7,284,664
2024-12-01	6,638,096

12 rows × 2 columns

Use the same idea with `UNION ALL` and a counter to build any series, such as 1 to 100 for bucketing or a list of working days for a schedule.

Preventing runaway recursion

If the data contains a loop (A reports to B, and B reports to A), a recursive query never ends. Two defences work well:

- **Stop by depth.** Add `WHERE level < 10` (or a similar limit) to the recursive member.
- **Use `UNION` instead of `UNION ALL`** when the same row could be reached twice, so the engine discards repeats.

::::warn Loops in the data A recursive query over a table with a cycle will run until the engine gives up or runs out of memory. Always check for cycles first, and always include a depth limit in production queries. ::::

::::tip Recursion versus window functions Recursion is for hierarchies of unknown depth. If the question is about a fixed number of steps (for example, "the manager's manager"), two self-joins are simpler and faster. ::::

::::key What to remember - A recursive CTE is an anchor, a recursive member that joins to itself, and a stopping point. - Carry depth and path columns down the recursion to report levels and breadcrumbs. - Walk upwards (to the top) or downwards (to all descendants), depending on the join direction. - Add a depth limit to protect against cycles. - Recursion can also generate series of numbers and dates, filling gaps in reports. ::::

Quick quiz

5 questions · tap an answer to check it

1. What is the role of the anchor member in a recursive CTE?

- **A** It joins the CTE to itself
- **B** It stops the recursion
- **C** It supplies the starting rows of the hierarchy
- **D** It removes duplicates

Answer: C. The anchor is the non-recursive first query, such as the top-level employees.

2. Which clause ends the recursion?

- **A** A LIMIT on the outer query
- **B** The recursive member returns no new rows
- **C** A HAVING clause
- **D** The ORDER BY clause

Answer: B. Recursion stops when a pass produces no rows.

3. Why is UNION ALL usually preferred in recursive CTEs?

- **A** It does not remove duplicates, so it is faster
- **B** It is the only syntax allowed
- **C** It sorts the rows
- **D** It guarantees no cycles

Answer: A. UNION ALL skips the duplicate-removal step; the trade-off is that cycles are not stopped automatically.

4. To list every month even when a month has no sales, you should:

- **A** Use an INNER JOIN with the sales table
- **B** Generate the months with a recursive CTE and LEFT JOIN the sales to it
- **C** Use DISTINCT on the order dates
- **D** Filter out zero sales

Answer: B. The generated series supplies the missing months, and the LEFT JOIN keeps them.

5. A query over employee data never finishes. The most likely cause is:

- **A** The table has too many columns
- **B** The data contains a cycle and the recursion has no stopping limit
- **C** The ORDER BY is missing
- **D** The anchor returns no rows

Answer: B. A cycle makes the recursive member produce rows forever.

EXERCISE **Build a reporting line report**

Using the org chart from Example 1, write a query that returns each person's name, their level, their direct manager's name, and the number of people who report directly to them. Order the result by level, then by name.

Show solution (worked answer)

Hint: join the `org` table to itself on `manager_id = emp_id` for the manager's name, and count direct reports with a separate `GROUP BY` on `manager_id`. Use the recursive `chain` CTE from Example 1 for the level.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.